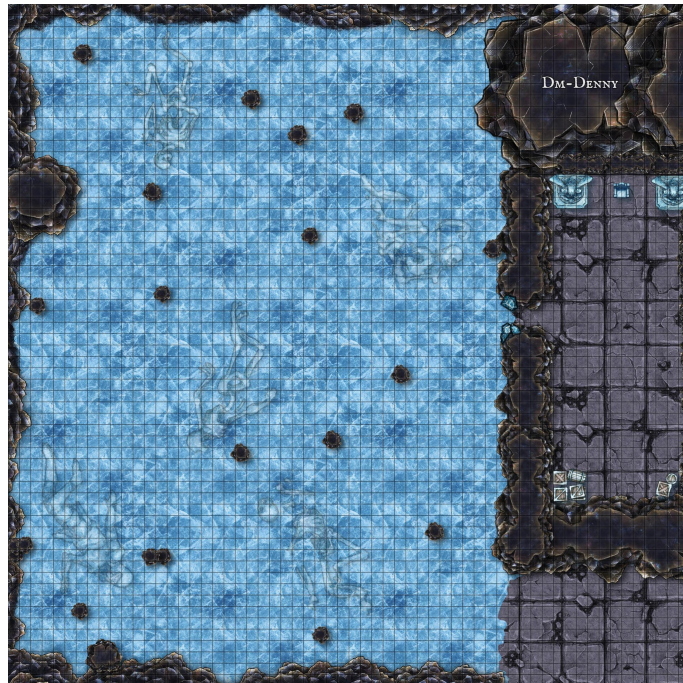


Tugas Kecil 3

Aplikasi Algoritma Pathfinding dalam Penelurusan Solusi dari Permainan *Ice Sliding Puzzle*

IF2211 Strategi Algoritma
Semester 2 Tahun 2025/2026



Disusun oleh:
Muhammad Aufar Rizqi Kusuma

Laboratorium Ilmu dan Rekayasa Komputasi
Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Daftar Isi

1	Pendahuluan	2
1.1	Deskripsi Tugas	2
1.2	Spesifikasi Program	3
1.3	Output	3
1.4	Spesifikasi Bonus	5
2	Algoritma Penyelesaian	7
2.1	Algoritma Uniform Cost Search (UCS)	7
2.2	Algoritma Greedy Best-First Search (GBFS)	7
2.3	Algoritma A*	8
2.4	Komparasi Algoritma <i>Pathfinding</i>	8
2.5	QnA	8
3	Implementasi	9
3.1	Arsitektur Program	9
3.2	Direktori Program	9
3.3	Modul-Modul Program	9
4	Eksperimen	10
4.1	Testing Kasus Dasar	10
4.2	Testing <i>Edge Cases</i>	10
4.3	<i>Post Mortem Analysis</i>	10
5	Penutup	11
6	Lampiran	12
6.1	Tautan GitHub	12
6.2	Checklist Tupil	12

Bab 1

Pendahuluan

1.1 Deskripsi Tugas

Tugas kecil ini berfokus pada pembuatan *solver* untuk permainan *Ice Sliding Puzzle*. Program menerima sebuah papan permainan berbentuk $N \times M$ dari berkas teks, kemudian mencari urutan gerakan yang membawa pin dari posisi awal Z ke tujuan 0. Selama pencarian, pin hanya dapat bergerak ke empat arah utama, dan karena lantai bersifat licin, pin akan terus meluncur sampai tertahan oleh dinding atau batu X.

Selain mencapai tujuan, solusi yang valid juga harus melewati angka-angka pada papan secara berurutan lengkap, mulai dari 0 hingga angka terbesar yang tersedia. Apabila pin melewati lava L, keluar dari batas papan, atau menginjak angka di luar urutan, maka keadaan tersebut dianggap gagal. Dengan demikian, program tidak hanya mencari jalur terpendek atau termurah, tetapi juga jalur yang mematuhi seluruh aturan permainan.

Gambar berikut memperlihatkan contoh papan permainan beserta legenda simbol yang digunakan pada laporan ini.

*	0	*	X	O
*	X	*	*	L
Z	*	1	*	*
*	*	*	X	*

Gambar 1.1: Contoh visualisasi papan *Ice Sliding Puzzle*.

Pada contoh di atas, simbol Z menandai posisi awal, 0 menandai tujuan, X adalah rintangan, L adalah lava, sedangkan 0 dan 1 adalah angka yang harus dilewati secara berurutan sebelum pin dapat dinyatakan menang. Tile bertanda * merepresentasikan jalur kosong yang dapat dilewati pin.

1.2 Spesifikasi Program

- Buatlah sebuah program tersebut dalam bahasa Java, Python, C++, C, atau GO yang mengimplementasikan algoritma Pathfinding (UCS, GBFS, dan A*) untuk mencari solusi dalam permainan Ice Sliding Puzzle Solver.
- Tugas dikerjakan berkelompok dengan anggota maksimal 2 orang, boleh lintas kelas maupun kampus. Silahkan isi pendataan kelompok pada link berikut.
- **Input:** program akan memberikan pengguna sebuah arahan pada terminal (jika tidak mengerjakan bonus) untuk memilih file test case berekstensi .txt, kemudian program membaca file test case tersebut yang berisi konfigurasi awal dari map Permainan yang akan diselesaikan. Perhatikan bahwa input dapat memiliki ukuran papan yang berbeda-beda (tidak harus persegi), serta program harus dapat memvalidasi apakah input yang diberikan merupakan input valid atau bukan.

Contoh perhitungan cost gerakan adalah sebagai berikut.

- Misalkan aktor melakukan satu gerakan ke kanan dan selama gerakan tersebut melewati tiga tile dengan cost berturut-turut: 2, 5, dan 4. Maka cost gerakan itu adalah $2 + 5 + 4 = 11$.
- Total cost solusi adalah jumlah dari semua cost gerakan yang dilakukan sepanjang solusi.
- Perlu diingat: tile 'X' dan 'L' tetap mempunyai nilai cost dalam input, tetapi tidak dihitung dalam solusi valid karena 'X' tidak pernah dilewati dan 'L' menyebabkan game over jika dilewati.

Contoh:

Gerakan 1: tiles traversed costs = 2, 5, 4, maka movement cost = 11

Gerakan 2: tiles traversed costs = 3, 2, maka movement cost = 5

Total cost solusi = $11 + 5 = 16$

1.3 Output

Program harus menghasilkan keluaran berikut setelah pencarian selesai:

1. Menampilkan solusi gerakan yang ditemukan (mis. RULUDR...).
2. Menampilkan total cost dari solusi.
3. Menampilkan visualisasi papan pada solusi, yaitu keadaan papan sehingga aktor dapat mencapai titik 0 menggunakan algoritma pathfinding yang dipilih. Visualisasi ini harus menandai lintasan akhir yang ditempuh aktor.
4. Menampilkan jumlah konfigurasi / iterasi yang ditinjau oleh algoritma. Program harus menyediakan opsi untuk menyimpan setiap iterasi (state) ke file berekstensi .txt sehingga dapat dianalisis atau diputar ulang nanti.

5. Menampilkan waktu eksekusi algoritma dalam milidetik. Pengukuran waktu harus meliputi hanya waktu komputasi (search), tidak termasuk waktu I/O membaca/menulis file.
6. Playback (bukan live update), dimana setelah algoritma selesai, program harus dapat memvisualisasikan proses pathfinding sebagai rangkaian langkah yang dapat diputar mundur/maju. Contoh playback di CLI dapat berisi fitur-fitur di bawah ini.
 - (a) Arrow kiri/kanan untuk mundur/maju satu step.
 - (b) ESC: prompt user untuk memasukkan nomor step X dan langsung lompat ke step tersebut.

Catatan: untuk mode GUI (bonus), playback harus memiliki kontrol kecepatan (play/pause/seek) dan kemampuan maju/mundur yang setara dengan pengalaman video. Berikut adalah contoh file .txt yang akan dijadikan sebagai input.

Baris pertama memuat 2 angka (N, M) yang menyatakan panjang dan lebar papan
N baris berikutnya merepresentasikan papan
N baris berikutnya merepresentasikan cost untuk melewati tile tersebut

Contoh Input

```
7 7
XXXXXXX
X0****X
X**X**X
X****OX
X***1LX
XZ**X*X
XXXXXXX
999 999 999 999 999 999 999
999 3 5 2 8 1 999
999 7 4 999 6 9 999
999 2 8 3 5 4 999
999 6 1 7 2 999 999
999 9 3 4 999 8 999
999 999 999 999 999 999 999
```

Keterangan:

- * = path yang bisa dilewati.
- X = Rintangan/Batu. Aktor akan berhenti tepat sebelum batu.
- L = Lava. Melewati lava akan mengalami game over (meskipun tidak berhenti tepat di lava).

- Z = Aktor/Pengguna.
- 0 = Titik tujuan.
- $\langle i \rangle$ = Angka. Ini adalah modifikasi pada spesifikasi tugas. Aktor harus melewati angka-angka ini sesuai urutan yang ada pada papan. Petak berangka 1 harus dilewati setelah aktor melewati petak berangka 0. petak berangka 2 harus dilewati setelah petak berangka 1 dan seterusnya (maksimal sampai 9).

Aturan tambahan yang penting.

- Jika aktor menginjak angka di luar urutan (mis. menginjak 1 sebelum 0), maka kondisi tersebut dianggap game over.
- Setelah sebuah petak angka dilewati sesuai urutan, petak tersebut berubah menjadi petak biasa dan dapat dilalui lagi tanpa batasan urutan.
- Petak angka adalah tile normal (bukan rintangan). Akibatnya, aktor akan meluncur melewatinya jika bergerak tanpa berhenti.
- Jika aktor bergerak menuju sisi papan (kanan/atas/kiri/bawah) dan tidak ada rintangan pada sisi tersebut untuk menghentikannya, maka aktor akan keluar papan dan kondisi dianggap game over. Dengan demikian, permainan harus diulang dari awal.
- Pin/aktor harus tepat berhenti pada titik 0 dan semua angka harus telah dilewati sesuai urutan agar kondisi dianggap sukses.
- Setiap tile memiliki cost traversal yang diberikan pada bagian kedua input. Cost merepresentasikan biaya ketika aktor melewati suatu tile.
- Cost gerakan dihitung dari jumlah cost seluruh tile yang dilalui selama satu gerakan sliding. Tile posisi awal pada awal gerakan tidak dihitung, tetapi tile tempat aktor berhenti pada akhir gerakan tetap dihitung.

1.4 Spesifikasi Bonus

Poin maksimal untuk bonus adalah 10. Mahasiswa dapat memilih satu atau beberapa spesifikasi bonus di bawah ini; total poin tambahan yang diberikan untuk tugas tidak boleh melebihi 10 poin.

1. **GUI penuh untuk program (maks 6 poin).** Membuat antarmuka grafis yang memungkinkan pengguna memilih file input, menampilkan papan, mengatur kecepatan playback, dan memutar langkah mundur/maju. Kriteria penilaian:
2. **Tambahkan algoritma pathfinding tambahan (maks 2 poin).** Implementasi setidaknya satu algoritma pathfinding lain selain yang diwajibkan (mis. Dijkstra, IDA*, BFS adaptif untuk sliding puzzle, atau algoritma lain yang relevan). Penilaian:

3. **Tambahkan dua alternatif heuristik (maks 2 poin).** Sediakan dua heuristik tambahan untuk A^* selain heuristik utama. Kriteria:
4. **Optimasi kecepatan pencarian (maks 5 poin, tetapi total bonus tetap dibatasi).** Optimasi yang dapat meningkatkan kecepatan menemukan solusi. Penilaian didasarkan pada bukti eksperimen dan metrik yang jelas.

Catatan penting tentang pemberian nilai.

- Total poin bonus yang diberikan tidak boleh melebihi 10, meskipun jumlah maksimum per-item jika dijumlahkan lebih besar. Dosen/TA akan memilih kombinasi fitur yang memberi nilai maksimal hingga 10 poin.
- Untuk setiap fitur bonus, sertakan bukti fungsionalitas: screenshot GUI atau rekaman pendek (untuk GUI), hasil benchmark (untuk kecepatan), dan contoh keluaran atau log yang menunjukkan algoritma/heuristik baru bekerja.
- Semua kode bonus harus disertakan pada repositori akhir dan dilengkapi instruksi build/run di README.

Instruksi pengumpulan bonus.

- Sertakan penjelasan singkat di README tentang fitur bonus yang diimplementasikan dan bagaimana mengaktifkannya (mis. argumen CLI atau flag pada GUI).
- Apabila mengumpulkan GUI, sertakan screenshot dan langkah untuk menjalankan (dependensi, command build, dan cara membuka file input).

Bab 2

Algoritma Penyelesaian

2.1 Algoritma Uniform Cost Search (UCS)

Uniform Cost Search (UCS) adalah algoritma pencarian yang mengembangkan node dengan biaya jalur terkecil $g(n)$ terlebih dahulu. UCS menjamin menemukan solusi dengan total biaya minimum apabila semua biaya tepi bernilai tidak negatif, karena selalu memilih state dengan g terkecil untuk diekspansi [1, 2]. Beberapa sifat penting UCS adalah sebagai berikut.

- **Optimalitas.** UCS optimal jika biaya tiap langkah non-negatif.
- **Keterlengkapan.** UCS akan menemukan solusi apabila ruang status berhingga dan ada path menuju tujuan.
- **Kompleksitas.** Bergantung pada branching factor dan distribusi biaya. Pada kasus terburuk dapat menimbulkan eksplorasi besar jika banyak node memiliki biaya jalur serupa.

2.2 Algoritma Greedy Best-First Search (GBFS)

Greedy Best-First Search (GBFS) merupakan algoritma pencarian yang memilih node untuk diekspansi berdasarkan nilai heuristik $h(n)$ saja, yakni node yang dianggap paling "dekat" ke tujuan menurut heuristik. GBFS tidak mempertimbangkan biaya jalur $g(n)$ sehingga dapat menjadi sangat cepat namun tidak menjamin optimalitas atau keterlengkapan dalam semua kasus [2]. Karakteristik GBFS dapat dijabarkan sebagai berikut.

- **Kecepatan.** Seringkali lebih cepat menemukan solusi dibanding pencarian yang mempertimbangkan $g(n)$, terutama bila heuristik sangat informatif.
- **Non-optimal.** GBFS tidak menjamin menemukan solusi dengan biaya minimal.
- **Sensitif terhadap heuristik.** kualitas heuristik sangat menentukan performa dan hasil pencarian.

2.3 Algoritma A*

A* (A-star) menggabungkan keuntungan UCS dan heuristik dengan menggunakan fungsi evaluasi

$$f(n) = g(n) + h(n),$$

di mana $g(n)$ adalah biaya jalur dari root ke node n dan $h(n)$ adalah heuristik estimasi biaya dari n ke tujuan. A* bersifat optimal jika heuristik h adalah *admissible* (tidak melebihi cost sebenarnya) dan akan efisien apabila h juga *consistent* (monotonik) [3, 2]. Sifat-sifat A* adalah sebagai berikut.

- **Optimalitas.** Terjamin bila h admissible.
- **Efisiensi.** Jumlah node yang diekspansi seminimal mungkin di antara semua algoritma yang menggunakan heuristik yang sama dan tetap optimal.
- **Ketergantungan heuristik.** Heuristik yang lebih informatif (mendekati nilai cost sebenarnya) akan mengurangi ruang pencarian.

2.4 Komparasi Algoritma *Pathfinding*

2.5 QnA

1. Definisi $f(n)$ dan $g(n)$ pada algoritma.
2. Apakah heuristik yang digunakan pada algoritma A* admissible
3. Secara teoritis, bandingkan algoritma yang diimplementasikan pada permasalahan ini

Bab 3

Implementasi

- 3.1 Arsitektur Program
- 3.2 Direktori Program
- 3.3 Modul-Modul Program

Bab 4

Eksperimen

- 4.1 Testing Kasus Dasar
- 4.2 Testing *Edge Cases*
- 4.3 *Post Mortem Analysis*

Bab 5

Penutup

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.

Bandung, 25 Maret 2026

Muhammad Aufar Rizqi Kusuma
13524061

Bab 6

Lampiran

6.1 Tautan GitHub

Link GitHub

6.2 Checklist Tupil

No	Poin	Ya	Tidak
1	Program berhasil dikompilasi tanpa kesalahan	☐	☐
2	Program berhasil dijalankan	☐	☐
3	Solusi benar dan mematuhi aturan permainan	☐	☐
4	Dapat membaca <code>.txt</code> dan menyimpan solusi ke <code>.txt</code>	☐	☐
5	Memiliki GUI	☐	☐
6	Ada algoritma pathfinding alternatif	☐	☐
7	Ada dua heuristik alternatif tambahan	☐	☐

Bibliografi

- [1] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.
- [2] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 3rd ed. Pearson, 2010.
- [3] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.